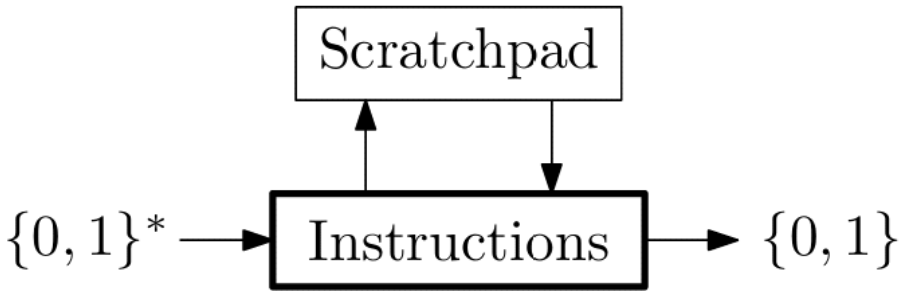
Computational_model

Saturday, March 28, 2026 12:44 PM

To judge the process complexity we need a standardize machine for running the algorithms

Our model will be the turing machine

How to Compute $f : \{0,1\}^* \to \{0,1\}$



- Every instruction proceeds by:
  - Reading a bit of the input;
  - Reading a symbol from the scratchpad;

  and based on that decide what to do next, namely:
  - either write symbol to the scratchpad, and proceed to another instruction;
  - or declare the computation finished, by stopping it and outputting either 0 or 1.

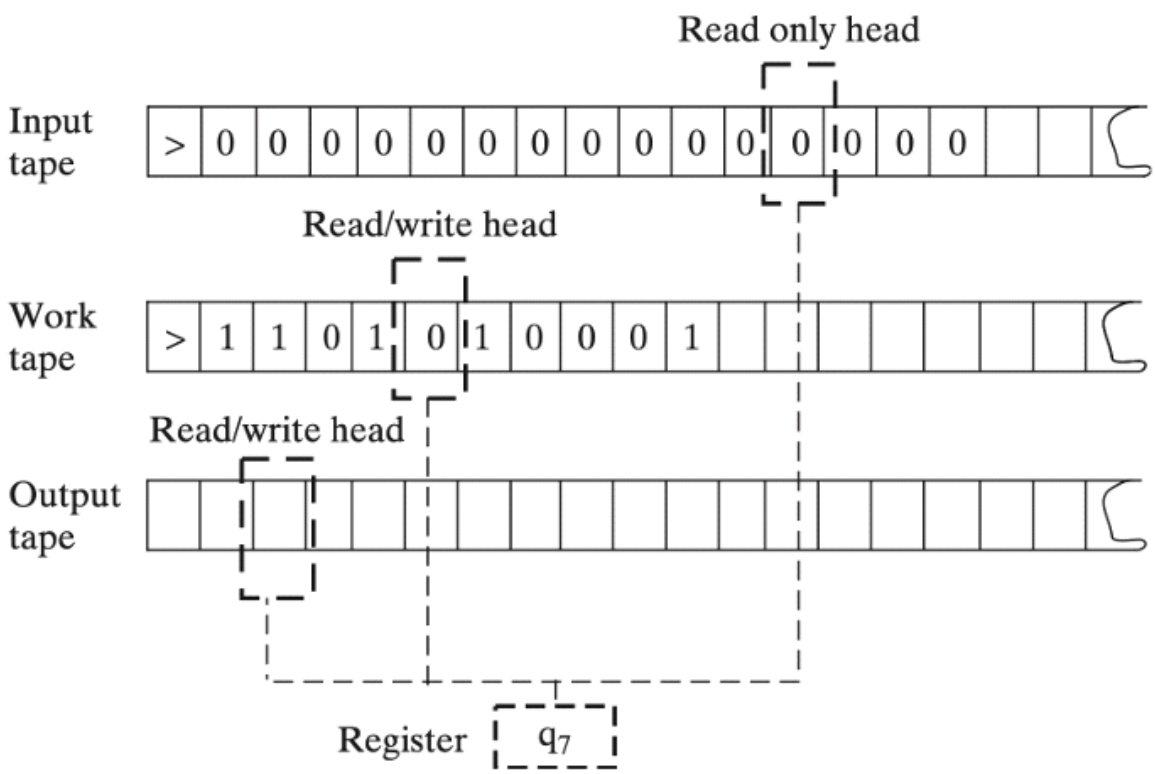
- The **running time** of this machine/process/algorithm on x is simply the number of these basic instructions which are executed on a certain input x.
- We say that the machine **runs in time** $T(n)$ if it performs *at most* $T(n)$ instructions on input strings *of length* n.

The Turing machine is a general machine. We can say low level machine or Bit level machine.
As a python program can be converted in Binary code. The same is for Complex turing machines.
Complex turing machine procedures/behaviours can be converted in binary code for a Generic Turing machine: called Universal turing machine

- Since there are finitely many instructions, machine descriptions can be **encoded as binary strings** themselves.

We can imagine Alfa as the code of our python program

- Given a string α, we indicate as $\mathcal{M}_\alpha$ the Turing Machine α encodes.
- There is a so-called **Universal** Turing Machine $\mathcal{U}$, which simulates any other Turing Machine given its string representation: from a pair of strings (x, α), the machine $\mathcal{U}$ simulates the behaviour of $\mathcal{M}_\alpha$ on x.
  - The simulation is very efficient: if the running time of $\mathcal{M}_\alpha$ were $T(|x|)$, then $\mathcal{U}$ would take time $O(T(|x|) \log T(|x|))$.



- It consists of k **tapes**, where a tape is an infinite one-directional line of cells, each of which can hold a simbol from a finite alphabel Γ, the *alphabet* of the machine.
- Each tape is equipped with **tape head**, which can read or write one symbol at a time *from* or *to* the tape. Each head can move left or right.
- Some of the tapes can be designated as **input tapes**, and are read-only.
- The last tape can be taken as the **output tape**, and in that case contains the result of the computation.
  - This slightly deviates from what we have said in our informal account, and is needed to compute arbitrary functions on $\{0, 1\}^*$.
  - The output tape(s) can be absent, and in that case the result can be taken as 0 or 1 depending on the *final state*.

- The machine has a finite set of *states*, called Q, which determine the action to be taken at the next step.
- At *each step*, the machine:
  1. **Read** the symbols under the k tape heads.
  2. For the $k - 1$ read-write tapes, **replace** the symbol with a new one, or leave it unchanged.
  3. **Change** its state to a new one.
  4. **Move** each of the k tape heads to the left or to the right (or stay in place).
- These instructions are of course very basic, and far from being close to the kind of instructions programming languages offer.

The Formal Definition

A **Turing Machine** (TM for short) working on k tapes is described as a triple (Γ, Q, δ) containing

- A finite set Γ of **tape symbols**, which we assume contains the *blank symbol* $\square$, the *start symbol* $\triangleright$, and the binary digits 0 and 1.
- A finite set Q of **states** which includes a designated *initial state* $q_{\texttt{init}}$ and a designated final state $q_{\texttt{halt}}$.
- A **transition function**

$$\delta : Q \times \Gamma^k \to Q \times \Gamma^{k-1} \times \{\texttt{L}, \texttt{S}, \texttt{R}\}^k$$

  describing the instructions regulating the functioning of the machine at each step.

- The **initial configuration** for the input $x \in \{0,1\}^*$ is the configuration $\mathcal{I}_x$ in which:
  - The current state is $q_{\texttt{init}}$.
  - The first tape contains $\triangleright x$, followed by blank symbols, while the other tapes contain $\triangleright$, followed by blank symbols.
  - The tape heads are positioned on the first symbol of the k tapes.

- A **final configuration** for the output $y \in \{0,1\}^*$ is any configuration whose state is $q_{\texttt{halt}}$ and in which the content of the output tape is $\triangleright y$, followed by blank symbols.

Delta: is a transaction function

- We say that $\mathcal{M}$ **returns** $y \in \{0,1\}^*$ **on input** $x \in \{0,1\}^*$ **in** t **steps** if

$$\mathcal{I}_x \xrightarrow{\delta} C_1 \xrightarrow{\delta} C_2 \xrightarrow{\delta} \ldots \xrightarrow{\delta} C_t$$

The following is an explanation of the formal definition of **computability** for functions within the framework of the theory of computation. It establishes the criteria for saying that a specific **Turing Machine (TM)** "solves" a mathematical task.

1. The Machine (M)

In this context, **M refers to a Turing Machine**, which is the universally accepted mathematical model for an abstract computer. A TM consists of a finite set of states and instructions (a transition function δ) that allow it to read and write symbols on infinite tapes. When we write $M(x)$, we are referring to the result produced by the machine after it starts with input x and follows its instructions until it reaches a final "halt" state (q_halt).

2. The Domain and Codomain ($\{0, 1\}^*$)

The notation $\{\mathbf{0}, \mathbf{1}\}^*$ represents the set of all possible finite, ordered sequences (strings) of binary digits.

- **The Task as a Function:** In complexity theory, almost any computational task—whether it's multiplying numbers, sorting a list, or checking graph reachability—is modeled as a function f that takes a binary string as an input and returns a binary string as an output.
- **Encoding:** If the original data (like a graph or a natural number) is not a string, it must be **encoded** into a binary string for the machine to process it.

3. The Condition ($M(x) = f(x)$)

The definition states that M computes f **if and only if ("iff")** the machine's output $M(x)$ exactly matches the intended result $f(x)$ for **every possible input** x in the domain.

- **Correctness:** It is not enough for the machine to work for some inputs; it must be correct for the entire infinite set of possible binary strings.
- **Termination:** For $M(x)$ to equal $f(x)$, the machine must eventually stop (halt). If a machine runs forever on a certain input, it does not "return" a value, and therefore cannot be said to compute a function for that input.

4. What is a "Computable" Function?

A function f is labeled **computable** simply if there exists at least one Turing Machine M that can satisfy the condition above

Efficiency and Runtime

- A TM $\mathcal{M}$ computes a function $f : \{0,1\}^* \to \{0,1\}^*$ **in time** $T : \mathbb{N} \to \mathbb{N}$ iff $\mathcal{M}$ returns $f(x)$ on input x in a number of steps smaller or equal to $T(|x|)$ for every $x \in \{0,1\}^*$. In this case, f is said to be **computable** in time T.
- A language $\mathcal{L}_f \subseteq \{0,1\}^*$ is **decidable** in time T iff f is computable in time T.

Remember a language is a classification function

Machines as Strings

This concept explains how a **Turing Machine (TM)**, which is a mathematical model of a computer, can be converted into a simple **binary string** (a sequence of 0s and 1s). This transformation is fundamental to computer science because it allows machines to be treated as **data** that can be processed by other machines.

Here is a detailed breakdown of the points mentioned in your query:

1. The Machine as a Finite Set of Instructions

A Turing Machine M is formally defined by three main components: its alphabet (Γ), its set of states (Q), and its **transition function (δ)**.

- The transition function δ is essentially the "program" or "rulebook" of the machine.
- It describes exactly what the machine should do at every step: based on its current state and the symbols it reads on its k tapes, it decides its next state, what symbols to write, and how to move the tape heads (L for left, S for stay, R for right).
- Because the sets of states and symbols are **finite**, the entire behavior of the machine is fully captured by the "graph" (or table) of this function δ.

2. Encoding the Machine (⌞ M ⌟)

Since the transition function δ is just a finite list of rules, we can use a **parsimonious encoding** (an efficient, standard way of formatting data) to turn that list into a single binary string, denoted as ⌞ M ⌟. This string is the "source code" of the Turing Machine.

3. The Two Necessary Conditions

For this encoding to be mathematically useful for further proofs (like the Halting Problem), it must satisfy two specific properties:

- **Condition 1: Every string represents a TM.** For any possible binary string $x \in \{0,1\}^*$, there must be a corresponding Turing Machine M. This ensures that we never "crash" because of a syntax error; even a completely random or "nonsense" string is interpreted as a valid (though perhaps useless) machine, such as one that halts immediately.
- **Condition 2: Infinite representations for every TM.** Every single Turing Machine M can be represented by **infinitely many different strings**. While there is only one **"canonical"** (standard) representation ⌞ M ⌟, you can create other versions of the same machine—for example, by adding "useless" instructions or padding the string with extra bits that don't change the machine's logic.

Why is this important?

This "Machines as Strings" concept leads to several critical results in the sources:

- **Universal Turing Machines (UTM):** We can build a single machine U that takes a string α (the description of machine M_α) and an input x, and then "runs" M_α on x. This is the theoretical basis for modern programmable computers.
- **Uncomputability:** Because machines are strings, we can try to feed a machine its own description as an input. This leads to the discovery of **uncomputable functions**, such as the function $uc(\alpha)$ or the **Halting Problem**, which cannot be solved by any possible algorithm.

Uncomputability

Theorem (Uncomputable Functions Exist)

There exists a function $uc : \{0,1\}^* \to \{0,1\}^*$ that is not computable by any Turing Machine.

$$uc(\alpha) = \begin{cases} 0 & \text{if } \mathcal{M}_\alpha(\alpha) = 1 \\ 1 & \text{otherwise} \end{cases}$$

This statement refers to the **constructive proof** that **uncomputable functions exist**. Instead of just

proving they must exist theoretically, the sources provide an explicit definition of such a function, denoted as $\boldsymbol{uc(\alpha)}$.

Breakdown of the Components

- $\boldsymbol{\alpha}$**:** This represents a binary string that encodes a specific Turing Machine (TM). The sources establish that every TM M can be represented by a string, and every string represents some TM.
- $\boldsymbol{M_\alpha}$**:** This is the specific Turing Machine that the string α encodes.
- $\boldsymbol{M_\alpha(\alpha)}$**:** This describes the process of running the Turing Machine M_α using its own encoding (α) as the input.
- $\boldsymbol{uc(\alpha)}$**:** This is the "uncomputable" function being defined. It returns **0** if the machine M_α outputs **1** when given its own code as input; otherwise, it returns **1**.

Why This Proves Uncomputability (The Contradiction)

The proof uses a technique called **diagonalization** to show that no Turing Machine can possibly compute this function uc:

1. **Assume uc is computable:** Suppose there exists a Turing Machine, let's call it M, that can compute uc. This means for any input string α, $M(\alpha) = uc(\alpha)$.
2. **Test M on its own code:** Since M is a Turing Machine, it has its own string encoding, which we can call $\llcorner M \lrcorner$. If we provide this code to M as an input, $M(\llcorner M \lrcorner)$ should output $uc(\llcorner M \lrcorner)$.
3. **The Contradiction:** Look at what happens according to the definition of uc:
   - If $\boldsymbol{uc}(\llcorner M \lrcorner) = \mathbf{0}$, then by definition, $M(\llcorner M \lrcorner)$ must equal **1**. But if M computes uc, $M(\llcorner M \lrcorner)$ should have been **0**.
   - If $\boldsymbol{uc}(\llcorner M \lrcorner) = \mathbf{1}$, then by definition, $M(\llcorner M \lrcorner)$ must **not** equal **1**. But if M computes uc, $M(\llcorner M \lrcorner)$ should have been **1**.

The sources summarize this impossibility as: $\boldsymbol{uc}(\llcorner M \lrcorner) = \mathbf{1} \Leftrightarrow \boldsymbol{M}(\llcorner M \lrcorner) \neq \mathbf{1}$ $\Leftrightarrow \boldsymbol{uc}(\llcorner M \lrcorner) = \mathbf{0}$. Because this leads to a logical contradiction (1 = 0), the initial assumption that uc is computable must be false.

$$halt(\llcorner(\alpha, x)\lrcorner) = \begin{cases} 1 & \text{if } \mathcal{M}_\alpha \text{ halts on input } x; \\ 0 & \text{otherwise.} \end{cases}$$

The halt function, is a function that takes in input the encoding of a program α and a string x, that will Be the input to the program α. The function is able to understand if the program α given in input x terminates in a finite time

Theorem (Uncomputability of *halt*)

The function halt is not computable by any TM.

A **Diophantine equation** is a specific type of mathematical equality defined by two main characteristics: it is a **polynomial equality** and it uses only **integer coefficients** with a **finite number of unknowns** (variables),.

The examples :

- $\boldsymbol{x\text{^}2 + 3y = 2x + 1}$: This equation uses variables x and y and integer coefficients like 3 and 2.
- $\boldsymbol{x\text{^}4 + 3x\text{^}3 + y - z = 12}$: This is a more complex polynomial with three unknowns (x, y, z) and integer coefficients.

Theorem (MDPR Theorem)

The problem of determining whether an arbitrary diphantine equation has a solution is undecidable.

Semantic Languages

A language $L \subseteq \{0,1\}^*$ is considered **semantic** if it focuses on the **function** a Turing Machine (TM) computes rather than the machine's specific implementation or code structure.

- **The Condition:** If two different Turing Machines, M and N, are functionally equivalent (meaning they produce the exact same outputs for the same inputs), a semantic language must treat them the same way. Formally: if M and N compute the same function, then $\llcorner M \lrcorner \in L$ if and only if $\llcorner N \lrcorner \in L$.
- **Extensional Properties:** These are properties of the "input-output" behavior of programs. For instance, "does this program eventually halt?" is a semantic property because it depends only on the function computed, not on how many states the machine has or what variable names are used.

Examples of Semantic Properties

The sources provide several examples of semantic languages:

- **Set of machines computing a specific function:** For example, the set of all programs that correctly sort a list.
- **Terminating programs:** The set of all programs that halt on all inputs (the Halting Problem is a classic example of a non-trivial semantic property).
- **Output constraints:** The set of all programs that never output a specific "bad" string.

By contrast, a **non-semantic** property would be something like "does the Turing Machine have more than 45 states?". This is an **intensional** property—two machines could compute the exact same function, but one might have 40 states and the other 50.

Undecidable

A language L is undecidable when we can't create a program ("decider") that can perfectly identify whether any other program belongs to L

In practical terms, there is **no substantial difference** between "undecidable" and "uncomputable," as both describe tasks for which no Turing Machine or algorithm can exist. However, the sources distinguish between them based on the **type of task** being described:

1. Uncomputable (Functions)

The term **uncomputable** is generally applied to **functions** ($f\colon A \to B$).

- A function is uncomputable if there is no Turing Machine that can take an input x and eventually halt with the correct output $f(x)$ for every possible x.
- The sources provide the example of the function $uc(\alpha)$, which is proved to be "intrinsically uncomputable by Turing Machines".

2. Undecidable (Languages and Decision Problems)

The term **undecidable** is specifically used for **languages** ($L \subseteq \{0,1\}$^) or **decision problems**.

- A decision problem asks a "Yes/No" question, such as "is this string x in the language M?".
- A language is **undecidable** if its **characteristic function** (the function that outputs 1 for "Yes" and 0 for "No") is **uncomputable**.
- For example, the **Halting Problem** is a decision problem, and the sources formally state that "the function $halt$ is not computable" and therefore the problem is **undecidable**.

Summary of the Relationship

According to the sources, **decidability is a special case of computability**:

- A task is a **decision problem** if it involves a boolean function (Yes/No).
- A language is **decidable** if and only if that boolean function is **computable**.
- Therefore, an **undecidable** language is simply one whose corresponding decision function is **uncomputable**.

the technical convention is to use **undecidable for Yes/No languages** and **uncomputable for general-purpose functions**.

Rice's Theorem

The theorem states: **Every semantic decidable language L is trivial**.

- **Triviality Defined:** A language is trivial if it is either empty ($L = \emptyset$) or contains every possible string ($L = \{0,1\}^*$).
- **The Logic:** If a property is semantic (it's about what the program does) and non-trivial (some programs have it and some don't), then it is **undecidable**. There is no algorithm that can look at the code of an arbitrary program and tell you if it possesses that property.
  So: **Every semantic and Not-Trivial language L is undecidable**.

1. The Logic of Rice's Theorem

The theorem states that **every semantic decidable language is trivial**. To understand the logic, you have to look at its three components:

- **Semantic (Extensional):** A property is semantic if it depends only on the **function** the machine computes, not how the code is written. If two different programs, M and N, always produce the exact same output for every possible input, a semantic property must treat them exactly the same.
- **Non-Trivial:** A property is trivial only if it applies to **all** programs ($L = \{0,1\}^*$) or **no** programs ($L = \emptyset$). If you can find even one program that has the property and one that doesn't, the property is **non-trivial**.
- **Undecidable:** If a property is both **semantic** and **non-trivial**, Rice's Theorem concludes it is **undecidable**. This means no algorithm exists that can take any arbitrary program's code and correctly determine if it has that property.

2. Practical Application: Explaining the Exercise

The example you cited comes from a specific exercise designed to classify languages. Let's look at the language $L_1 = \{ \llcorner M \lrcorner \mid M(001)\ terminates\ with\ output\ 110 \}$, (all the machines that given in input 001 return as outpit 110):

- **Is it semantic? Yes.** The property "outputs 110 when given 001" is purely about the **input-output behavior** of the machine. If you have two different machines that both compute a function where $f(001) = 110$, both machines will be in L_1. The internal states or the number of lines of code don't matter—only the result does.
- **Is it trivial? No.** We can easily find a machine that is in L_1 (e.g., a simple machine that ignores its input and always writes "110") and a machine that is not (e.g., a machine that always outputs "000"). So Becouse L_1, it is neither empty nor universal, it is non-trivial.
- **Conclusion:** Because L_1 is a **non-trivial semantic property**, Rice's Theorem tells us it is **undecidable**.

NOTE:
you can easily write a program M that, when given "001", outputs "110". However, Rice's Theorem is

not about the difficulty of writing M; it is about the impossibility of writing a **decider**—a single program that can look at the code of *any* arbitrary program and tell you if it has that property.

How to Prove a Problem to be *Computable*?

Method 1: create a Turing machine that solves the problem

- One can of course **construct a TM** which computes the given function or that decides the given language.

Method 2: write the pseudo code of the algorithm. The operation of the algorithm must be elementary

- One can describe, in a more informal way, **an algorithm** which itself computes the function or decides the language.
  - It is of course crucial to be sure that all steps the algorithm perform, i.e., all instructions, are elementary, or at least compute functions which are already known to be computable.

How to Prove a Problem to be *Uncomputable*?

Method 1: Show that there is a way to convert the strings of our problem is strings of problem that we know that hare uncomputables

- You can prove that the problem under consideration, call it $\mathcal{L}$ is **at least as hard** as a problem you already know to be undecidable, call if $\mathcal{G}$
  - You have to show that there is a computable way ϕ of turning strings in $\{0,1\}^*$ into strings in $\{0,1\}^*$ in such a

Method 2: use Rice's Theorem

EXERCISES

Exercise 1:

provide a proof for the **undecidability of the Halting Problem**. The halting problem asks whether a given Turing Machine M_α will eventually stop (halt) when run on a specific input x, or whether it will run

forever.
The exercise proves that a function $halt(\llcorner (\alpha, x) \lrcorner)$, which returns 1 if $M_\alpha(x)$ terminates and 0 otherwise, is **not computable** by any Turing Machine.

The Proof Strategy: Reduction to uc

To proof the uncomputability. We use **the function uc** (the "uncomputable" function). Our plan is to implement $\boldsymbol{uc}$ using $halt$. In other words we whant to use $halt$ as part of the code that implement $\boldsymbol{uc}$. If we manage to implement $\boldsymbol{uc}$ using $halt$, that means that also $halt$ is uncomputable becouse it is impossibile to implement uc

1. **Assume a solution exists:** Suppose there is a hypothetical Turing Machine, M_{halt}, that can successfully compute the $halt$ function for any input.
2. **Construct a new machine:** Now we can build a new machine called M_{uc} designed to compute the uc function. Using M_{halt} as a "subroutine"
3. **The uc function definition:** The function uc(α) is defined as returning 0 if $M_\alpha(\alpha) = 1$, and returning 1 otherwise (including if it diverges or returns something else). It is a known result in computability theory that this function is uncomputable.

How the Constructed Machine (M_{uc}) Works

```
def M_uc(α):
    x = M_halt(α,α)
    if x == 0:
        return 1
    else:
        b = μ(α,α)
        return not(b)
```

On an input string α (which represents a machine description), M_{uc} performs the following steps:

- **Step 1: Check for Halting.** It calls the hypothetical M_{halt} on the pair (α, α). This checks if the machine M_α will halt when given its own description as input.
- **Step 2: Handle Non-Halting.** If M_{halt} returns 0 (meaning $M_\alpha(\alpha)$ never terminates), M_{uc} can safely return 1.
- **Step 3: Handle Halting.** If M_{halt} returns 1 (meaning $M_\alpha(\alpha)$ is guaranteed to terminate), M_{uc} uses a **Universal Turing Machine** (μ) to simulate the computation of $M_\alpha(\alpha)$ and get the result, b.
- **Step 4: Return the Negation.** M_{uc} then returns the negation of b (if $b = 1, it$ returns 0; if b=0, it returns 1).

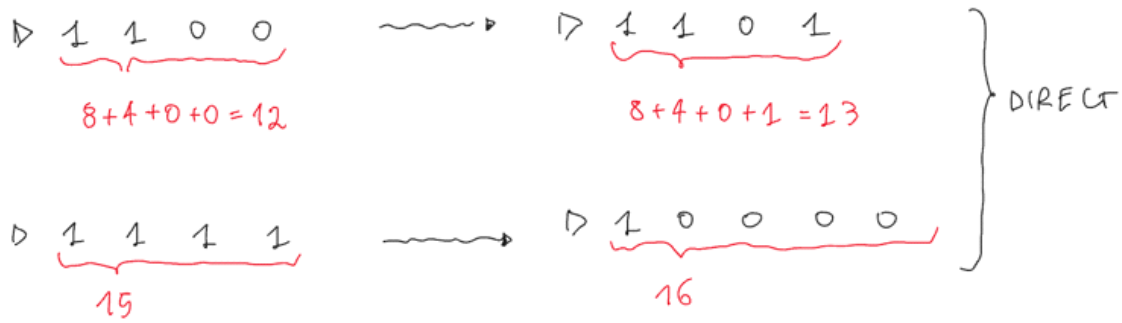
Conclusion

The construction of M_{uc} ensures that it correctly computes the uc function. However, since we already know that **the function uc is uncomputable**, such a machine M_{uc} cannot possibly exist. Because M_{uc} cannot exist, the "subroutine" it relies on—the hypothetical machine M_{halt}—must also not exist. Therefore, the Halting Problem is undecidable.

Exercise 2

Is about defining a Turing Machine named Inc. that taken as input a string: it sum 1 to that string and return it as output. $\boldsymbol{Inc(x) = x + 1}$

The goal is to show that this operation is linear. So we need to create a machine that to this task in linear time



If we try to directly read from left to right the number to add the 1 we had the problem of the carry bit. Because the carry is moving right to left. In other words the problem is that we have no way to know where to leave the space for the carring bit, and at some point we will need to invert the direction of the tape for bring the carring bit back.

The solution is to read the input from right to left (reverse the input). In these way, we can easily move the carry

This is the formal definition of the Turing Machine:

Q is the set of the States:
q_{init} is the initial state,
q_{halt} is the final state
q_0 means that we still have to add the +1 to the number
q_1 Means that we have already add the +1 to the number

Γ is the alphabet of the tapes and include the starting simbol, the empty space simbol, 0 and 1

δ is the set of all the transitions
Ex on how to read a transition:
$(q_0, 0) \rightarrow (q_1, 1, \text{L})$
If we are in the state q_0 and we read a 0 in the tape.
We change to state q_1 and we write 1 on the tape and we move the tape left (in this particular machine when the operation of adding 1 is complete the machine go back to the start position

before terminating)

SOLUTION

$Q = \{q_{init}, q_0, q_1, q_{halt}\}$

$\Gamma = \{\triangleright, \square, 0, 1\}$

δ:

$(q_{init}, \triangleright) \mapsto (q_0, \triangleright, R)$

$(q_0, 0) \mapsto (q_1, 1, L)$

$(q_0, 1) \mapsto (q_0, 0, R)$

$(q_0, \square) \mapsto (q_1, 1, L)$

$(q_1, 0) \mapsto (q_1, 0, L)$

$(q_1, 1) \mapsto (q_1, 1, L)$

$(q_1, \triangleright) \mapsto (q_{halt}, \triangleright, S)$

$(q_{halt}, \triangleright) \mapsto (q_{halt}, \triangleright, S)$

Exercise 3

Exercise 3 asks whether a specific function, $halt_{star}$, is computable or uncomputable and requires a proof for the claim.

Definition of the Function $halt_{star}$

The function $halt_{star}$ takes as input a sequence $S = \langle \alpha, x_1, \dots, x_k \rangle$, where $\boldsymbol{\alpha}$ is the encoding of a **single Turing Machine** and $x_1, \dots, x_k$ is a sequence of k inputs for that machine. The function is defined as follows:

- **Returns 1:** If the machine M_α terminates on **all** the provided inputs ($M_\alpha(x_1)$ terminates, ..., and $M_\alpha(x_k)$ terminates).
- **Returns 0:** If there exists **at least one** input x_i (where $1 \leq i \leq k$) such that $M_\alpha(x_i)$ does not terminate.

The Claim: Uncomputable

The solution states that this function is **uncomputable** (or undecidable).

The Proof: Reduction from the Halting Problem

To prove that $halt_{star}$ is uncomputable, the sources use a **reduction** from the classic **Halting Problem** (L_{halt}), which is already known to be undecidable.

1. **Define a Reduction Function (Φ):** We must define a computable function Φ that transforms any input for the Halting Problem into a valid input for $halt_{star}$.
2. **The Mapping:**

The "Mapping" is essentially a **translator**.

- **The Goal:** We want to show that if we had a program to solve $halt_{star}$, we could use it to solve the Halting Problem.
- **The Input for Halting:** A standard Halting Problem input consists of a machine α and a single input x (written as $\langle \alpha, x \rangle$).
- **The Input for $halt_{star}$:** This function expects a machine α and a *sequence* of inputs $x_1, \ldots, x_k$.
- **The Translation:** The function Φ takes the pair $\langle \alpha, x \rangle$ and simply repackages it as a $halt_{star}$ input where the sequence of inputs has a length of exactly one ($k = 1$).
  - So, $\Phi(\langle \alpha, x \rangle) = \langle \alpha, x \rangle$, where the first x is the input for the machine α.
- **Why it works:** This mapping is **computable** because it does nothing more than slightly adjust the string encoding to fit the expected format of the other function.

3. **Logical Equivalence:**
   The "Equivalence" proves that our "translator" doesn't change the answer to the problem. We need to show that: $\langle \alpha, x \rangle \in L_{halt} \Leftrightarrow \Phi(\langle \alpha, x \rangle) \in L_{halt_{star}}$
   - **Left side (L_{halt}):** By definition, $\langle \alpha, x \rangle$ is in L_{halt} if and only if **machine M_α terminates on input x**.
   - **Right side ($L_{halt_{star}}$):** By definition, a sequence is in $L_{halt_{star}}$ if the machine **terminates on ALL inputs** in the provided sequence.
   - **The Link:** Since our mapped sequence only contains **one** input (x), the condition "terminates on all inputs in the sequence" simplifies to just "**terminates on x**".

Conclusion of the Proof

Because the two problems now give the exact same answer for these inputs, we have shown that $halt_{star}$ is **at least as hard** as the Halting Problem. Since the Halting Problem has already been proven to be **uncomputable** (meaning no Turing Machine can ever solve it), it is logically impossible for $halt_{star}$ to be computable. If it were, you could just "translate" any Halting Problem into it and solve the unsolvable.

Exercise 4

Exercise 4 involves classifying three languages according to their decidability or undecidability. Here is the solution:

$L_1 = \{ \alpha \mid M_\alpha(001) \textit{ terminates with output } 110 \}$

- **Classification: Undecidable**.
- **Reasoning:** This language represents a **semantic (extensional) property** because it depends solely on the function computed by the Turing Machine (its input/output behavior) rather than its internal structure. Since this property is **non-trivial**—meaning there are Turing Machines that satisfy it (e.g., a machine that always outputs 110) and those that do not—**Rice's Theorem** dictates that the language is undecidable.

L_2 = { α | M_α is a Turing machine with at most 5 states }

- **Classification: Decidable**.
- **Reasoning:** This is an **intensional (structural) property** of the machine's description rather than its computed function. To decide if a machine belongs to L_2 , one only needs to parse its encoding α and count the number of states listed in its finite set of states Q or its transition function δ. Because the description is a finite string, this check is always possible and finite.

L_3 = { (x, z) | $M_x(z)$ does not terminate before the third

computation step}

- **Classification: Decidable**.
- **Reasoning:** Although this involves the termination of a machine, it is restricted to a **finite and constant number of steps** (the third step). To decide this, a **Universal Turing Machine (UTM)** can be used to simulate M_x on input z for exactly three steps. If the machine has not reached its final state (q_{halt}) by the end of these steps, it belongs to the language. Since the simulation is bound by a fixed number of steps, it is guaranteed to terminate and provide an answer.

Exercise 5

Consider the problem given by:

- **Input:** a string $x \in \{0,1\}^*$
- **Output:** the reversed string $x^R \in \{0,1\}^*$

1. **Describe (informally) a TM solving the problem. What is its (time) complexity?**
2. Same but we want a TM with **only 1 tape**.

Solution for Part 1 (Two-Tape Turing Machine)

The first solution utilizes a Turing Machine with **two tapes**: one for reading the input and one for writing the output. The process is as follows:

- **Step 1:** On the input tape, move the reading head to the right until it reaches the blank symbol (□). This process takes $\boldsymbol{n+1}$ **steps**, where n is the length of the string.
- **Step 2:** Move the input tape head back to the left (to read the string in reverse) and move the output tape head to the right (to begin writing).
- **Step 3:** Until the blank symbol (□) is reached on the input tape, the machine reads a bit from the input tape, writes that same bit onto the output tape, and then moves the input head left and the output head right. This copying process takes $\boldsymbol{n}$ **steps**.

Time Complexity: The total execution time for this machine is $2n + 1$, which simplifies to $\boldsymbol{O(n)}$.
Formal Definition: The machine uses an alphabet Γ = {△ , □, 0, 1, } and specific states to manage the logic:

- $\boldsymbol{q_{wb}}$: "Go right on input tape until I read □."
- $\boldsymbol{q_c}$: "Copying a bit of input to output."

Solution for Part 2 (One-Tape Turing Machine)

With only one tape, the problem can be solved as follows:

- **Step 1:** Go to the right until the blank symbol (□) is reached, and overwrite it with a placeholder (e.g., λ). This takes $O(n)$ time.
- **Step 2:** Go to the left until the first bit of the input is reached. This takes $O(n)$ time.
- **Step 3:** Perform the following loop (*****):
  - Go to the right until a character that is not λ is found.
  - **If the bit is 0:** Replace it with λ, move to the right until the first blank (□) is found, overwrite it with 0, then return to the first λ and repeat the loop (*****).
  - **If the bit is 1:** Follow the same process as above, but write a 1 instead.
- **Step 4:** Continue this process (scanning and moving bits) until the entire string is

processed.

- The notes indicate a variation where you go to the right until □ is reached, then go back to the left, read the bit, replace it with λ, move to the right to the last □, and write the bit there.

Time Complexity: Because the head must scan back and forth across the tape for each bit of the input, the machine runs in $\boldsymbol{O(n^2)}$ time. The source notes that "reversing a tape content only is a polynomial overhead."

Exercise 6

designing a **one-tape Turing Machine (TM)** capable of computing a function that **flips all bits of an input string**.

Problem Definition

- **Input:** A binary string.
- **Goal:** Change every 0 to a 1 and every 1 to a 0.

The Solution

The TM is defined with the following components:

- **Alphabet (Γ):** $\{0, 1, \square\}$, where □ represents the blank symbol.
- **States (Q):** The machine uses three states: q_{start},q_{flip}, and q_{accept}.
- **Transition Logic (δ):**
  - The machine begins in q_{start} and moves to q_{flip}.
  - While in the $\boldsymbol{q_{flip}}$ **state**, the machine performs the following loops:
    - If it reads a **0**, it writes a **1** and moves the head to the **right**.
    - If it reads a **1**, it writes a **0** and moves the head to the **right**.
  - Once the machine encounters a **blank symbol (□)**, signifying the end of the input string, it transitions to the $\boldsymbol{q_{accept}}$ **state**.

Complexity

The machine processes each bit of the input exactly once. Therefore, for an input of length n, the TM runs in $\boldsymbol{n + 1}$ **steps**, resulting in a time complexity of $\boldsymbol{O(n)}$.

Exercise 7

Following the sequence from the exercise session sources, **Exercise 3** involves designing an efficient **2-tape Turing Machine (TM)** to recognize a specific language.

Problem Definition

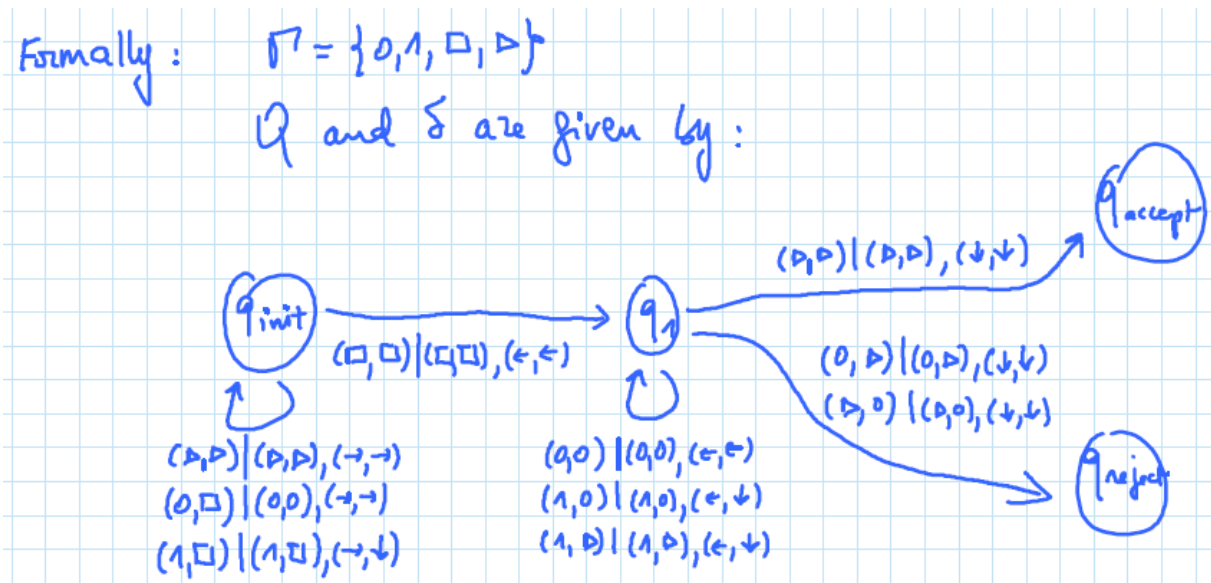
- **Task:** Define a TM that accepts the language L = $\{w \in \{0,1\}^* \mid w \text{ has the same number of } 0's \text{ and } 1's\}$.
- **Constraint:** The solution must specifically use a **2-tape** model.

The Solution

The sources describe an evolution from a basic 3-tape idea to an optimized 2-tape process:

- **Preliminary Idea (3 tapes):** One could use Tape 1 for input, Tape 2 to copy all '0's, and Tape 3 to copy all '1's. After reading the entire input, the machine would check if Tape 2 and Tape 3 contain the same number of symbols.

- **Optimized Solution (2 tapes):** To use only two tapes, the machine uses the second tape as a dynamic "balance" tracker.
  1. **Tape 1** is the read-only input tape.
  2. **Tape 2** acts as the scratchpad.
  3. **Operation:**
     - As the machine reads the input on Tape 1, if it encounters a **0**, it writes a symbol on Tape 2 and moves the head to the **right**.
     - If it encounters a **1**, it moves the head on Tape 2 to the **left**.
  4. **Acceptance:** Once the machine reaches the end of the input (the blank symbol □ on Tape 1), it verifies the position of the head on Tape 2. If the head has returned to the **start symbol (△)**, it indicates that the number of 1s perfectly balanced the number of 0s, and the machine **accepts**.



Complexity

Because the machine scans the input exactly once and performs constant-time head movements for each bit, the time complexity is $\boldsymbol{O(n)}$.

Exercise 8

the task is to **define a Turing Machine (TM) that decides the language L_1**, which consists of all binary strings ($w \in \{0,1\}^*$) containing an **odd number of 0's and an even number of 1's**.

The Solution

The solution models the TM's behavior using four primary states that track the parity (even or odd) of the digits processed so far:

- $\boldsymbol{q_{ee}}$ **(Start State):** Even number of 0's and even number of 1's.
- $\boldsymbol{q_{oe}}$ **(Target State):** Odd number of 0's and even number of 1's.
- $\boldsymbol{q_{eo}}$**:** Even number of 0's and odd number of 1's.
- $\boldsymbol{q_{oo}}$**:** Odd number of 0's and odd number of 1's.

State Transitions

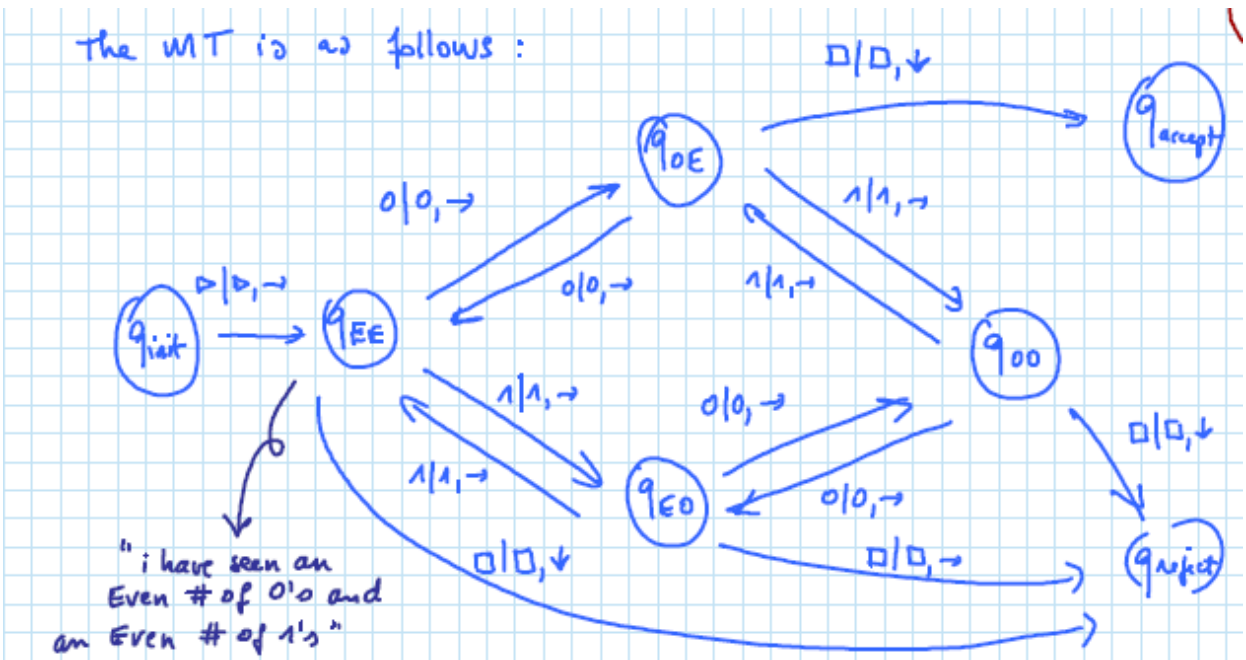
As the TM reads each character, it moves between these states:

- **Reading a '0'** flips the parity of the 0's:
  - $q_{ee} \leftrightarrow q_{oe}$
  - $q_{eo} \leftrightarrow q_{oo}$.
- **Reading a '1'** flips the parity of the 1's:
  - $q_{ee} \leftrightarrow q_{eo}$
  - $q_{oe} \leftrightarrow q_{oo}$.

Acceptance and Rejection

When the machine encounters a blank symbol (□), signifying the end of the input:

- If it is in the $\boldsymbol{q_{oe}}$ **state** (meaning it has seen an odd number of 0's and an even number of 1's), it transitions to $\boldsymbol{q_{accept}}$.
- If it is in any other state (q_{ee}, q_{eo}, or q_{oo}), it transitions to $\boldsymbol{q_{reject}}$.



Formal Components

The sources also specify the formal components of this TM:

- **Alphabet (Γ):** $\{0, 1, \square\}$.
- **States (Q):** $\{q_{ee}, q_{oe}, q_{eo}, q_{oo}, q_{accept}, q_{reject}\}$.
- **Transition Function (δ):** Maps each (state, symbol) pair to a (new state, symbol, move), such as $(q_{ee}, 0) \rightarrow (q_{oe}, 0, R)$.

Exercise 9

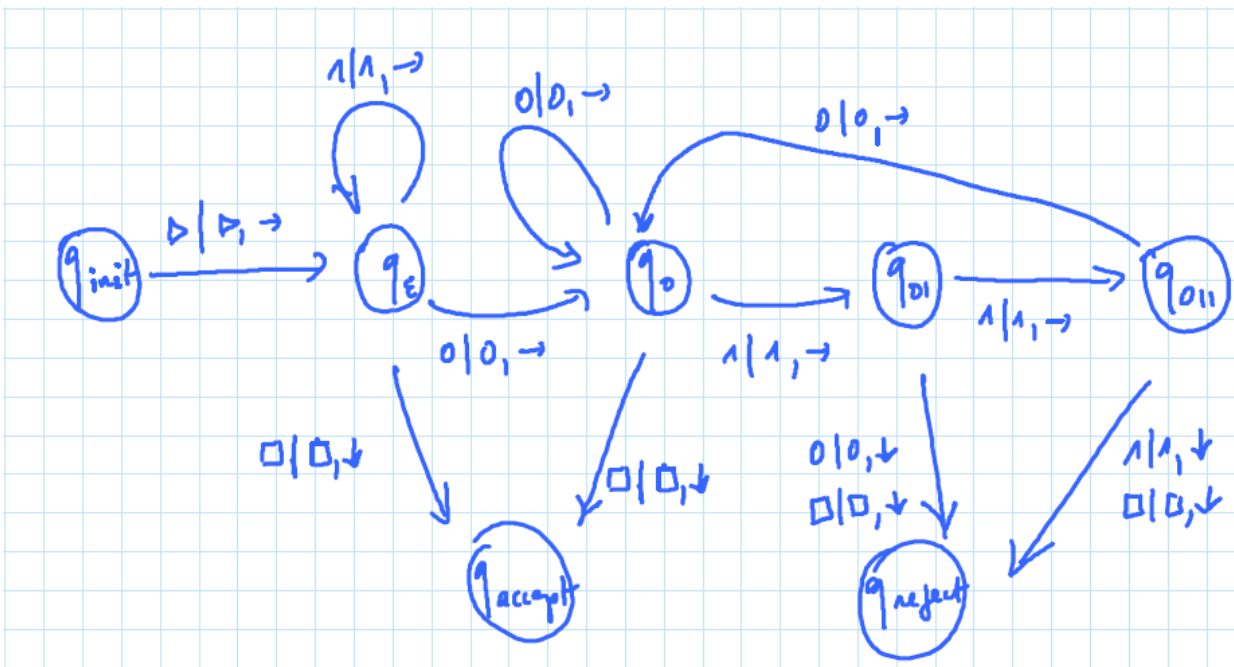
the objective is to define a Turing Machine (TM) that decides the language L_2. This language consists of all binary strings where **every occurrence of the sequence "01" is followed by the sequence "10"**. This effectively means that every time "01" appears, it must be part of the larger pattern "**0110**".

The Solution

The solution uses a state machine to scan the input string from left to right, ensuring that the "01" to "10" requirement is never violated.

State Transitions

- q_{start} **(Initial State):** The machine stays in this state as long as it reads **1**s. If it reads a **0**, it moves to q_1, as it might be the start of a "01" pattern.
- q_1 **(Found '0'):** If the machine reads another **0**, it stays in q_1. However, if it reads a **1**, it has found the sequence "**01**" and moves to q_2.
- q_2 **(Enforcing '10' - Step 1):** Having just seen "01," the machine now requires the next two characters to be "1" then "0."
  - If it reads a **1**, it moves to q_3.
  - If it reads a **0** or a **blank (□)**, the rule is violated, and the machine moves to q_{reject}.
- q_3 **(Enforcing '10' - Step 2):**
  - If it reads a **0**, the "0110" pattern is complete, and the machine returns to q_{start} to continue scanning.
  - If it reads a **1** or a **blank (□)**, the rule is violated, and the machine moves to q_{reject}.



Acceptance and Complexity

- **Acceptance:** The TM accepts the string if it reaches the end of the input (a blank symbol □) while in state q_{start} or q_1, as these states indicate that no unfinished or illegal "01" patterns exist.
- **Time Complexity:** Because the machine reads each character exactly once and transitions based on a fixed set of rules, it runs in $O(n)$ time.

NOTE: RICE's THEOREM AND EXERCISE 9

Applying **Rice's Theorem** to the L_2 language from Exercise 9 requires a distinction between the language itself and properties of the languages recognized by Turing Machines.
Based on the sources, here is how the theorem relates to L_2:

1. L_2 is Decidable

As demonstrated in Exercise 9, L_2 is a set of binary strings where every "01" is followed by "10". The sources provide a specific Turing Machine that **decides L_2 in $O(n)$ time**. Because a TM exists that can correctly accept or reject any string w in finite time, L_2 is a **decidable language**.

2. Rice's Theorem Does Not Apply to L_2 Directly

Rice's Theorem does not apply to languages of strings like L_2; it applies to **properties of languages**

recognized by Turing Machines. Specifically, it states that any **non-trivial semantic property** of a TM's language is undecidable.

- **Semantic property:** A property that depends only on the language recognized by the TM, not its internal code or implementation.
- **Non-trivial:** A property that is true for some TMs and false for others.

3. Applying Rice's Theorem to a Property Based on L_2

If you were to create a new language, let's call it $L_{property}$, consisting of the encodings of all Turing Machines whose recognized language is exactly L_2 ($L_{property}$ = {⟨ M ⟩ | $L(M) = L_2$}), then Rice's Theorem would apply:

- **It is Semantic:** Whether $L(M) = L_2$ depends only on the behavior of the TM, not how it is written.
- **It is Non-trivial:** There are some TMs that recognize L_2 (like the one in Exercise 2) and some TMs that recognize other languages (like a machine that accepts all strings).

Therefore, while L_2 itself is decidable, the problem of **determining if an arbitrary Turing Machine recognizes L_2 is undecidable** by Rice's Theorem.

RECAP: REDUCTION

a **reduction** is a method used to compare the relative difficulty of two computational problems. Formally, a reduction from a language L_A to a language L_B (denoted as $\boldsymbol{L_A \leq L_B}$) is a **computable function** R that transforms an input w into a new input $R(w)$ such that:

- If w is in the first language ($w \in L_A$), then $R(w)$ must be in the second language ($R(w) \in L_B$).
- If w is not in the first language, then $R(w)$ must not be in the second language.

Key Concepts of Reduction

- **"At Least as Hard As":** The notation $L_A \leq L_B$ indicates that L_B is at least as hard to solve as L_A. If you have a way to solve (or decide) L_B, you can use the reduction function to solve L_A as well.
- **Mechanism:** To check if a string w belongs to L_A, you first apply the reduction to get $R(w)$ and then check if $R(w)$ belongs to L_B.
- **Computability:** The transformation function R itself must be "computable," meaning it can be performed by a Turing Machine in a finite amount of time.

Exercise 10

the goal is to explore **reductions** between two languages related to the halting problem:

- $\boldsymbol{L_h^{\epsilon}}$: The set of encodings ⟨ M ⟩ of Turing Machines that halt on the **empty string (ϵ)**.
- $\boldsymbol{L_h^{00}}$: The set of encodings ⟨ M ⟩ of Turing Machines that halt on the **input "00"**.

The exercise demonstrates that these two problems are equally difficult by showing they can be reduced to one another.

1. Reduction $\boldsymbol{L_h^{\epsilon} \leq L_h^{00}}$

To prove this, we define a computable function R that transforms an encoding of a machine M into an encoding of a new machine M' such that M halts on ϵ if and only if M' halts on "00".
M is the machine for the language $\mathbf{L_h^{\epsilon}}$
M' is the machine for the language $\boldsymbol{L_h^{00}}$

- **Construction of $\boldsymbol{M'}$:** Given an input, M' is designed to **erase the first two symbols of its input** and then behave exactly like M.
- **Logic:** If we give M' the input "00", it will erase the "00", leaving the tape empty, and then run M on that empty tape. Therefore, M' halts on "00" if and only if M halts on ϵ.

- **Computability:** This reduction is computable because creating M' simply involves prepending a small, finite amount of code (to handle the erasure) to the existing code of M.

2. Reduction $L_h^{00} \leq L_h^{\epsilon}$

This reduction works in the opposite direction, transforming a machine M into a machine M' such that M halts on "00" if and only if M' halts on ϵ.
M is the machine for the language $\boldsymbol{L_h^{00}}$
M' is the machine for the language $\mathbf{L_h^{\epsilon}}$

- **Construction of M':** M' is designed to **write "00" on the first two cells of its tape** and then move its head back to the start of the tape to simulate M.
- **Logic:** If M' is started on an empty input (ϵ), its first action is to create the string "00" on the tape. It then executes M on that string. Thus, M' halts on an empty input if and only if M halts on "00".
- **Formal Transition:** The sources illustrate the internal states for M' to achieve this: it transitions from a start state to write the first '0', then the second '0', and finally returns to the beginning of the tape before launching the original machine M.

Summary of the Solution

The exercise concludes that both reductions are valid because the transformation functions are **computable**. By showing $L_h^{\epsilon} \leq L_h^{00}$ and $L_h^{00} \leq L_h^{\epsilon}$, we establish that the two languages are computationally equivalent in terms of their decidability.

Exercise 11

This exercise focuses on classifying different languages of Turing Machine encodings based on whether they are **trivial**, **semantic**, and **decidable**. This exercise serves as a practical application of **Rice's Theorem**, which states that any non-trivial semantic property of a language recognized by a Turing Machine is undecidable

Understanding the Checks

- **Trivial:** Is the property true for *every* possible Turing Machine (TM) or for *no* TM at all? If you can find even one machine that has the property and one that doesn't, the property is **non-trivial**.
- **Semantic:** Does the property depend only on the **language** the machine recognizes (what it does)? If two machines recognize the exact same language but one has the property and the other doesn't, the property is **not semantic**. (a language must have only an Input/output property to be semantic)
- **Decidable:** Can we build a TM that takes the code of machine M as input and always tells us "yes" or "no" in a finite amount of time? (this TM is a machine that should recognize if the machine M is part of the machine described by the language)

L1: M halts on all inputs

- **Trivial? No.** Some TMs halt on everything (like a machine that accepts immediately), while others loop forever on everything.
- **Semantic? Yes.** Whether a machine halts or loops on a specific input defines its behavior and the language it recognizes.
- **Decidable? No.** Because this is a **non-trivial semantic property**, Rice's Theorem tells us it is **undecidable**.

L2: M accepts the empty input (ϵ)

- **Trivial? No.** The property is non-trivial because it is easy to construct a machine that accepts every input in exactly 42 steps and another machine that does not (for example, one that accepts in 5

steps or one that never accepts).

- **Semantic? No.**
  This property is **not semantic** because it depends on the specific execution details (the number of steps) rather than the language recognized.
  - The sources provide a counterexample: Consider two machines, $\boldsymbol{M_1}$ and $\boldsymbol{M_2}$.
  - $\boldsymbol{M_1}$ is designed to accept all inputs after only **5 steps**.
  - $\boldsymbol{M_2}$ is designed to perform 42 steps of computation (perhaps some "useless" movements) and then accept.
  - Both machines recognize the **same language** (the set of all strings), but $\boldsymbol{M_1} \notin \boldsymbol{L_2}$ and $\boldsymbol{M_2} \in \boldsymbol{L_2}$.
  - Since the property differentiates between machines that recognize the same language, it is not a semantic property.
- **Decidable? No.** The professor say that there is a reduction to the halting problem but has not demostrated it

L3: $\boldsymbol{M}$ accepts more than 42 inputs

- **Trivial? No.** There are machines that accept nothing, machines that accept 10 strings, and machines that accept an infinite number of strings.
- **Semantic? Yes.** The number of strings in the language is a property of the language itself, not the code.
- **Decidable? No.** This is another non-trivial semantic property, Rice's Theorem tells us it is **undecidable**.

L4: $\boldsymbol{M}$ reject $\boldsymbol{0^{42}}$

- **Trivial? No.** This property is non-trivial because there are some Turing Machines that reject the string 0^{42} and many others that do not.
- **Semantic? Yes.** This is a semantic property because it depends entirely on the **language** recognized by the machine (specifically, whether the string 0^{42} is in the set of rejected strings). It does not depend on the internal structure or the number of states in the machine's code.
- **Decidable? No.** Because this is a **non-trivial semantic property**, it is **undecidable** according to **Rice's Theorem**.

L5: On input x, $\boldsymbol{M}$ writes on the 42nd cell of the tape

- **Trivial? No.** There are machines that perform operations only on the first few cells and others that use a large amount of the tape.
- **Semantic? No.** This is **not a semantic property** because it concerns the internal operation (tape usage) rather than the final language recognized. The sources explain that you could have two machines, M_1 and M_2, that both recognize the same language (for example, they both accept the empty string immediately). However, M_1 might accept in one step without moving, while M_2 might perform 42 "useless" steps to reach and write on the 42nd cell before accepting. Because they recognize the same language but one has the property and the other doesn't, it is not semantic.
- **Decidable? yes.** The professor say it is decidable but he hasn't explain way.

L6: $\boldsymbol{M}$ has more than 42 states

- **Trivial? No.** Turing Machines can be constructed with any number of states.
- **Semantic? No.** this is a **structural property** of the machine's code. One can build a very complex machine with 100 states or a simplified version with 10 states that both recognize the same language.
- **Decidable? Yes.** This is **decidable** because a "decider" only needs to **inspect the code** of the machine M and count the number of states defined in its transition function.

L7: $L(M)$ is finite

- **Trivial? No.** Some Turing Machines recognize finite languages (a specific list of strings), while others recognize infinite languages.
- **Semantic? Yes.** Finiteness is a property of the **language** itself, not the machine's internal structure.
- **Decidable? No.** Because this is a non-trivial semantic property, **Rice's Theorem** dictates that it is **undecidable**.

L8: { $w = \{0,1\}^*$ | if $w = \llcorner M \lrcorner$ then M accept ϵ or M reject ϵ }

- **Trivial? Yes.** Every machine accept the empty string or reject the empty string, and because every string can be a machine this language include all the strings.
- **Semantic? Yes.** This property describes a specific input requirement
- **Decidable? Yes.** Because it is both semantic and trivial the **Rice's Theorem** tell us that it is decidable